

HoopHub

BookGameSeat



Sommario

1	Software Requirement Specification	3
1.	Introduction.....	3
2.	User Stories.....	4
3.	Functional Requirements.....	4
4.	Use Cases.....	5
2	Storyboards	5
3	Design	6
4	Testing	8
5	Code	8
6	Video	8
	Notes.....	8

Final Project Detailed Instructions

1 Software Requirement Specification

1. Introduction

1.1 Aim of the document

1.2 Overview of the defined system

1.3 HW e SW requirements

1.4 Related systems (at least 2), Pros and Cons.

2. User Stories

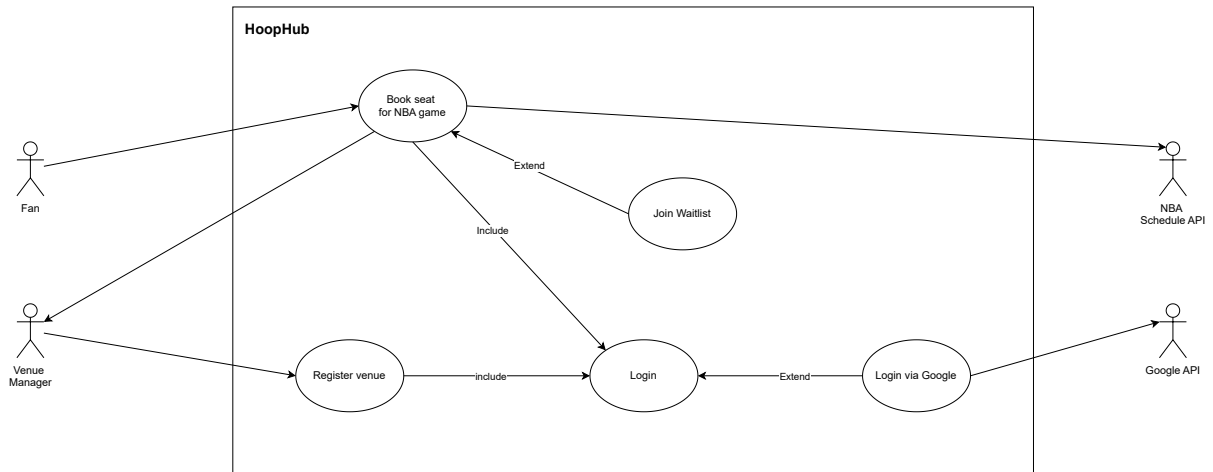
- 2.1 As a registered fan, I want to book a seat in a venue for my favorite team's NBA game, so that I can get a reservation.
- 2.2 As a guest user, I want to view the schedule of the next two weeks NBA games, so that I can know when matches will take place.
- 2.3 As a venue manager, I want to register my venue with name, address, accessibility and available channels, so that fans can discover it and send booking request.

3. Functional Requirements

- 3.1 The system shall provide users authentication via username and password.
- 3.2 The system shall provide a form to request a seat reservation for an NBA game in a venue.
- 3.3 The system shall provide a form to register a new venue.

4. Use Cases

1. Use Case Diagram



2. Internal Steps - Book Seat for NBA game

Precondition: Fan / Venue Manager is logged in.

Main Success Scenario

1. The fan requests to book a seat for an NBA game.
2. The system retrieves the game from the NBA Schedule API.
3. The system displays the list of NBA games scheduled for the next 4 weeks.
4. The fan selects a specific game.
5. The system retrieves and displays available venues for that game.
6. The fan selects a venue.
7. The system displays the venue details* and requests confirmation.
8. The fan confirms the booking request.
9. The system registers the booking request with status* pending and generates a pre-confirmation code*.
10. The system notifies the Venue Manager about the new booking request.
11. The Venue Manager reviews and accepts the booking request.
12. The system updates the request status* to confirmed and notifies the Fan.

Extensions (Alternative Scenario)

3a. No games available: The system informs the Fan that no upcoming games are currently available. Use case ends.

5a. No venues available for the selected game: The System informs the fan that no venues are currently offering that game and suggests choosing another game. Use case resumes at step 3.

6a. Fan applies filters to refine the venue list: The System updates and displays the filtered results based on the selected criteria.

7a. Selected venue is fully booked: The System informs the Fan that the venue is full and offers the option to join the waitlist via use case "Join Waitlist". Use case ends.

8a. Fan cancel before confirming the booking request: The System cancels the operation and returns the fan to their home page. Use case ends.

11a. Venue Manager rejects the booking request: The System updates status* to rejected and notifies the fan. Use case ends.

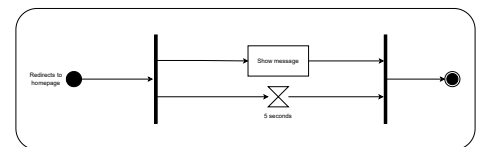
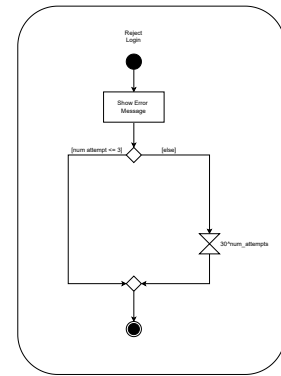
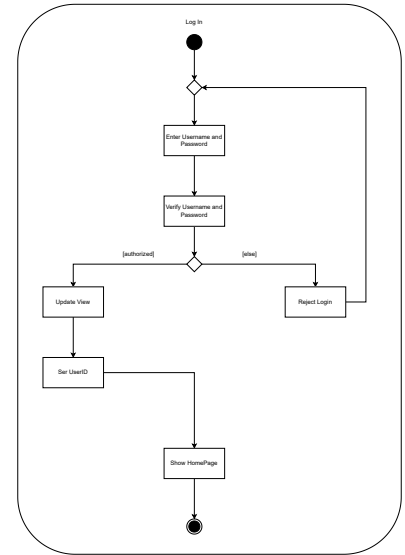
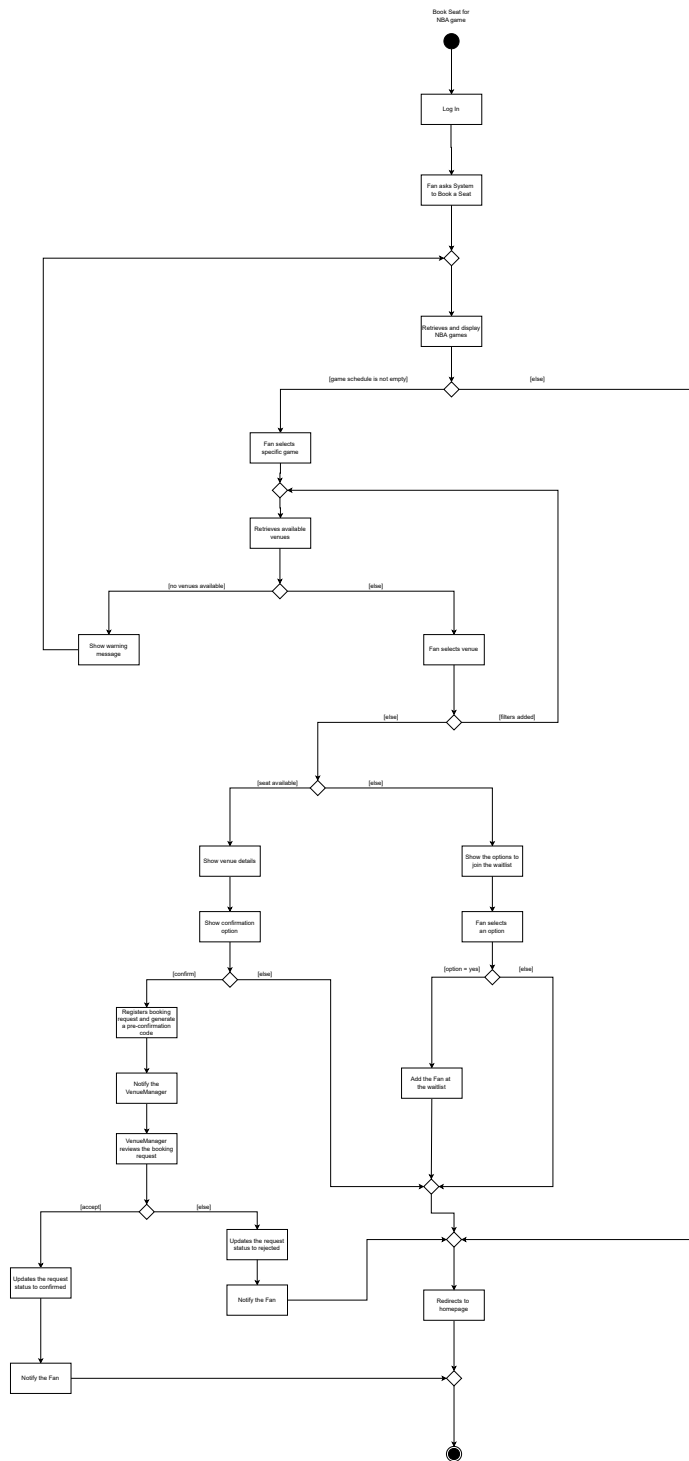
2 Storyboards

Quantity: min 2 screens per member, covering all the major functionalities described in SRS, developed using Draw.io or similar

3 Design

- Class diagram:
 - 1 VOPC per member.(analysis)
 - 1 design-level diagram per member (e.g. that includes patterns, or specific solutions that improve the engineering level of the system)
- Design patterns: 1 different pattern per member. Possibly try to apply the pattern within the context of the project.

3.3 Activity Diagram



- Sequence diagram: 1 per member.
- State diagram: 1 per member.

4 Testing

- Develop at least 3 test cases per person. In each test (class) file, please report (via Java comments) the name of the person in charge.

5 Code

- ~4K LOC
- Similar functionality implemented with two GUIs.
- Exceptions: at least 2 per member (do not just catch and back-propagate the exceptions, but properly handle them. Possibly define your own error logic by means of exceptions)
- Be able to show that Svn(or Git) + SonarCloud is correctly installed in one of your computer and it is able to analyze your project for rule violations. No rule must be violated (no smells, no vulnerabilities, no bugs). This will be checked during the exam.
- The application has to run in 2 different modalities: demo-version, full-version
 - o in the demo-version the application provides all the functionalities you decided to support BUT all the data are only stored IN-MEMORY, NOTHING has to be stored on any persistency layer. In this sense, as all the entities are kept in memory, if the application is turned off all the data are lost
 - o in the full-version the application provides all the functionalities you decided to support AND the data can be stored on some persistency layer. Further detail in the next requirement.
- At least one DAO shall be provided in two versions DMBS and file system.

6 Video

- A 1 to 2 minutes recorded video with audio of the developed system performing the expected functionalities. *.mpeg

Notes

1. System.out* CLI-related smells are allowed.
2. Deliverables should be compliant with HMW constraints.
3. Code must be compliant with deliverables. Discrepancies can exist and they should be pointed out in such artifacts.
4. The project is one per team. The grade is shared across team members. All team members must submit the entire project.
5. The project submission procedure will be communicated (only) to the students subscribed to a specific exam date.